PROOF OF CONCEPT · TEST QUALITY

Mutation Testing POC

A short walkthrough of what we measured, what we learned, and how easily we can adopt it

Java 17 · Spring Boot · PITest

Angular 21 · TypeScript · StrykerJS

November 2026

WHAT WE HAVE ACCOMPLISHED SO FAR

Two real projects, 716 mutants analysed

716

Total mutants

710 backend, 6 frontend

13%

Backend score

92 of 710 mutants killed

38%

Test strength

92 of 241 mutants tests reached

50%

Frontend score

3 of 6 killed, one service file

Test improvement opportunities in both stacks

These include a backend test that mocks the class under test, which line coverage alone could not reveal.

WHAT LINE COVERAGE CANNOT SHOW

Line coverage versus mutation coverage

Backend packages under com.bnpparibas.blank

Package	Line coverage	Mutation coverage	Test strength
utils	69%	73%	89%
controllers	49%	26%	53%
services.impl	76%	20%	29%
beans	87%	12%	19%
gen.api.dto	19%	8%	58%

> For example, in beans 87% of lines execute while 12% of mutants are killed, so line coverage alone gives a more favourable picture than mutation testing.

ASSERTION ANALYSIS

Which assertions mutation testing can check

Mutator	What it changes	Assertion gap it reveals
Return values	Replaces the return value	Returned value is not asserted
Conditionals / boundaries	> to >=, == to !=	Branch and edge cases not asserted
Math operators	+ to -	Result is not verified
Void call removal	Removes the call	Side effects are not verified
String / boolean literals	'dark' to empty, false to true	Initial state and constants not asserted
equals() / hashCode()	Alters equality logic	Equality contract not tested

> 18 weakness categories are catalogued in the architects' deck, each with the mutator, why the test survives, and the fix.

BACKEND EXAMPLE · JAVA 17 · PITEST

Backend: mocking the class under test

getAllBranches() runs a SQL query and returns the branches. PITest replaced its body with an empty list, and the existing test did not detect the change.

Mutant injected

```
// PITest: empty-object return
List<Branch> getAllBranches() {
  return Collections.emptyList();
}
```

Existing test (mutant not detected)

```
when(blankService.getAllBranches())
  .thenReturn(expectedList); // mocks the SUT
result = blankService.getAllBranches();
assertEquals(expectedList, result);
```

Suggested fix: mock only the dependency

```
when(jdbcTemplate.query(any(), any()))
  .thenReturn(rows); // real service runs
result = blankService.getAllBranches();
assertEquals(expectedBranches, result);
```

Observation: the mutant survived

The assertion is made against the mock, so the real method is not exercised. With the suggested fix, the test detects the empty-list mutant.

FRONTEND EXAMPLE · ANGULAR 21 · STRYKERJS

Frontend: asserting outcomes, not only calls

theme.service.ts · setDarkTheme() — 6 mutants: 3 killed, 2 survived, 1 with no coverage.

Two mutants that survived

```
// BooleanLiteral: false → true
_darkTheme = new BehaviorSubject(true);
// StringLiteral: 'dark' → ''
isDark ? '' : 'light'
```

Existing test (mutants not detected)

```
jest.spyOn(service._darkTheme, 'next');
service.setDarkTheme(true);
expect(service._darkTheme.next)
  .toHaveBeenCalled(); // call only
```

Suggested fix: assert the outcomes

```
expect(service._darkTheme.value).toBe(false);
service.setDarkTheme(true);
expect(document.body.getAttribute(
  'coras-theme')).toBe('dark');
```

Observation: both mutants survived

27 tests cover this code; adding assertions for the initial value and the DOM attribute would detect both.

HOW THE TOOLS WORK

How PITest and Stryker create mutants

PITest 1.25.7

Java · bytecode mutation · HTML + XML reports

Mutates compiled .class files in memory, so no recompile per mutant

Collects coverage first, then runs only the tests that cover the mutated line

Mutants run in forked JVMs for isolation

StrykerJS 10.1.1

TypeScript · source (AST) mutation · interactive HTML

Parses the source into an AST and mutates the tree

All mutants live in one instrumented copy, switched on one at a time

Runs only the Jest tests that cover each mutant; a type checker drops mutants that do not compile

PITest mutates what the JVM runs. Stryker mutates what the developer wrote. The reports read the same way: killed, survived, no coverage.

ADOPTION EFFORT

Adoption effort: configuration, not code

Backend: one Maven plugin

Setting	Value
Plugin	pitest-maven 1.25.7, junit5 1.2.3
Target	com.bnpparibas.blank.*
Output	HTML + XML reports
Run	mvn pitest:mutationCoverage

Frontend: one npm package

Setting	Value
Tool	StrykerJS 10.1.1 + Jest
Scope	src/app/core/services/**/*.ts
Thresholds	High 80% · Low 60%
Run	npx stryker run

No code changes

Production code and existing tests untouched

One tool per stack

Plugin or package plus a small config

Scope to start

One package or folder keeps runs short

NEXT STEPS

Proposed next steps, for your consideration

1

Address the high-value gaps first

Services, equals/hashCode and mappers first

2

Add changed-code checks to CI

Run on changed code per pull request, full scan nightly

3

Raise the threshold gradually

Start at today's baseline and raise it each quarter

With your kind support, we would like to extend this POC to the critical services